Feuille de démo — de Hugging Face à une API gouvernée

Durée indicative : 15 à 20 minutes, hors installation des opérateurs. Deux sessions séparées : **ADMIN dans une fenêtre normale**, **DATA SCIENTIST dans une fenêtre privée**. Garder cette feuille sur un autre écran. Les intitulés ont été relevés dans la sandbox RHOAI 3.5.2 le 5 novembre 2026.

Situation : « Notre équipe Data Science veut utiliser un assistant sans administrer son serveur. Je suis l'administrateur : je vais publier le modèle, donner un accès limité à l'équipe, puis vérifier sa consommation. »

Avant le direct : `make prepare` sur une sandbox neuve, GPU libre, compte `demo-user` prêt, mots de passe disponibles localement, `make endpoints` pour les liens. Si `make demo` a déjà été exécuté, les objets existent : rejouer leurs créations nécessite un reset préparé avant le direct. Ne pas tenter de créer deux objets portant le même nom.

1 — ADMIN : entrer dans OpenShift AI et créer le projet

À dire : « Je commence dans OpenShift avec mon compte administrateur. Mon projet va regrouper le déploiement du modèle. »

1. Se connecter à la console OpenShift de la sandbox avec le compte admin.
2. En haut, ouvrir le lanceur d'applications, puis **Red Hat OpenShift AI**. Si le lien n'est pas proposé dans cette version, ouvrir l'URL OpenShift AI donnée par `make endpoints`.
3. Dans OpenShift AI, ouvrir **Projects** → **Create project**.
4. Champ **Name** : `maas-demo`. Vérifier que le nom de ressource est aussi `maas-demo`.
5. Champ **Description** : `Un assistant partagé pour l'équipe Data Science`.
6. Cliquer **Create** et ouvrir le projet.

Petit geste technique à expliquer : « La gateway accepte les routes des projets autorisés. » Après la création UI, depuis le terminal admin :

```
oc label namespace maas-demo maas.opendatahub.io/gateway-access=true \
  app.kubernetes.io/part-of=maas-demo --overwrite
```

Preuve à montrer : le projet `maas-demo`, encore sans modèle.

2 — ADMIN : déployer un vrai modèle Hugging Face

À dire : « Hugging Face contient les poids du modèle. OpenShift va les télécharger et lancer le moteur d'inférence sur notre GPU. »

Dans **Projects** → **maas-demo** → **Deployments**, cliquer **Deploy model**.

Écran Preconfigure deployment

- **Project** : `maas-demo` si demandé.
- **Next**.

Écran Model details

- **Model location** : `URI`.
- **URI** : `hf://Qwen/Qwen2.5-0.5B-Instruct`.
- Décocher **Create a connection to this location** pour cette source publique sans secret. Le message indiquant que la connexion ne sera pas sauvegardée est attendu.
- **Model type** : `Generative AI model (Example, LLM)`.
- **Next**.

À dire : « Nous choisissons un petit modèle instruct public pour réduire le chargement. Le même principe s'applique à des modèles plus grands, avec les ressources et accès nécessaires. »

Écran Model deployment

- **Model deployment name** : `demo-llm`.
- **Description** : `Assistant de démonstration — Qwen public servi sur GPU`.
- **Deployment method** : **LLM inference service with llm-d**.
- **Hardware profile** : **MaaS - 1 GPU**, installé par le kit. Sur la sandbox historique, `gpu-profile` est aussi disponible, mais ses valeurs par défaut diffèrent.
- **Customize resource requests and limits** : CPU demandé `1`, limite `4` ; mémoire demandée `4 GiB`, limite `8 GiB` ; GPU `1`.
- **Replica count** : `1`.
- **Next**.

À dire : « Je réserve un GPU et une seule réplique. Le moteur sert ensuite toutes les requêtes autorisées par MaaS. »

Écran Advanced settings

- Cocher **Publish as MaaS**.
- **Require token authentication** devient décoché et désactivé dans cet assistant : l'authentification passe par la gateway MaaS. Ne pas créer de token de service account à la place de la clé MaaS.
- **Add as AI asset endpoint** : optionnel pour le Playground ; inutile pour l'appel API montré ici.
- Cocher **Add custom runtime environment variables**, puis **Add variable**.
- Nom : `VLLM_ADDITIONAL_ARGS`.
- Valeur : `--dtype=half --max-model-len=2048 --gpu-memory-utilization=0.60 --enforce-eager`.
- **Next**.

Écran Review

Lire à voix haute : projet `maas-demo`, URI Qwen, une réplique, profil GPU, **MaaS endpoint: Yes**. Cliquer **Deploy model** lorsque l'on joue la création sur un projet vide.

À dire pendant l'attente : « Le premier démarrage récupère les images et les poids. Ce travail de préparation ne sera pas refait à chaque appel du data scientist. »

Preuve : revenir dans **Projects** → **maas-demo** → **Deployments** et attendre **Ready**. Le contrôle technique est :

```
oc rollout status deployment/demo-llm-kserve -n maas-demo --timeout=900s
oc get llminferenceservice,maasmodelref -n maas-demo
```

Transition : « Le modèle tourne et sa publication MaaS existe. Je définis maintenant qui peut le consommer et à quelle limite. »

3 — ADMIN : créer la souscription

Ouvrir **Settings → MaaS governance → Subscriptions → Create subscription**.

- **Name** : `demo-quota`.
- **Description** : `Équipe Data Science — 2 000 tokens par minute`.
- **Priority** : `10`. Expliquer seulement si nécessaire : elle intervient dans la sélection entre plusieurs souscriptions accessibles ; nos clés seront liées explicitement à leur souscription.
- **Groups** : sélectionner **demo-team**.
- **Models → Add models** : chercher `demo-llm`, vérifier le projet **maas-demo**, cliquer **Add model** sur sa ligne, puis **Add models** dans la fenêtre.
- Sur la ligne du modèle, cliquer **Add token limit**.
- Dans **Edit subscription token limits** : saisir **2000** tokens per **1**, puis choisir **minute** dans la liste (le défaut peut être hour).
- **Save**.
- Décocher **Create authorization policy** pour montrer l'autorisation séparément à l'étape 4.
- **Create subscription**.

À dire : « Cette souscription rassemble les bénéficiaires, le modèle et sa limite de consommation. Un token est une unité de texte traitée par le modèle, pas une requête. »

Preuve : la souscription apparaît avec `demo-team`, `demo-llm` et `2,000 / 1 minute`. La gouvernance complète devient disponible après l'autorisation.

Variante courte : garder **Create authorization policy** coché crée aussi une policy associée. Montrer alors cette policy, sans ajouter `demo-access` par-dessus. La suite des scripts du kit utilise `demo-access` ; pour la variante courte, réconcilier les noms avant un test de suppression de policy.

4 — ADMIN : créer l'autorisation

Dans **MaaS governance → Authorization policies**, utiliser **Create authorization policy** si le formulaire est proposé. Nom `demo-access`, groupe `demo-team`, modèle `demo-llm` du projet `maas-demo`.

Pour un geste reproductible dont les champs correspondent exactement à la ressource, la console OpenShift permet aussi **+ → Import YAML**. Remplir le document suivant puis **Create** :

```
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: demo-access
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: demo-team
  modelRefs:
    - name: demo-llm
      namespace: maas-demo
```

À expliquer en remplissant : `namespace` désigne le tenant de gouvernance ; `subjects.groups` contient les bénéficiaires ; `modelRefs` désigne le modèle publié et son projet.

À dire : « La souscription fixe le cadre de consommation. L'autorisation donne le droit d'atteindre ce modèle. Je conserve les mêmes groupes et modèles des deux côtés. »

Preuve : policy Active/Ready dans l'interface, puis `MaaSModelRef/demo-llm` Ready.

Pour l'affichage du modèle côté utilisateur dans cette build, appliquer une fois les droits de découverte du kit :

```
oc apply -f manifests/govern/user-rbac.yaml
```

5 — DATA SCIENTIST : découvrir son accès et créer une clé

Ouvrir la session data scientist : garder la fenêtre normale connectée en admin. Dans le menu du navigateur, choisir **Nouvelle fenêtre de navigation privée** (ou **InPrivate**), puis ouvrir l'URL OpenShift AI donnée par `make endpoints`. Cette fenêtre possède une session indépendante ; un simple nouvel onglet normal partage la session admin.

- Sur l'écran de connexion, choisir le fournisseur contenant `demo-user` : **corp-demo** sur cette sandbox ; sur une nouvelle sandbox, utiliser celui préparé par le kit.
- **Username** : `demo-user`. **Password** : le mot de passe conservé localement dans `secrets/demo-user.password`.
- Cliquer **Log in**, puis vérifier que le compte affiché en haut à droite est bien **demo-user**.

À dire : « Je passe maintenant dans la session du data scientist pour montrer les accès que l'administrateur lui a accordés. »

Montrer la souscription et créer la clé :

1. **Gen AI studio → API keys**.
2. Onglet **Subscriptions**, puis cliquer **demo-quota** : montrer le modèle et la limite `2,000 / 1 minute`.
3. Dans cette fiche, **Create API key**.
4. **Subscription** est préremplie avec **demo-quota** ; vérifier le modèle et son quota.
5. **Name** : `demo-notebook` ; **Description** : `Mon premier appel à l'assistant`.
6. **Expiration → On date** : choisir une date couvrant la démonstration, par exemple J+7. Le champ accepte `AAAA-MM-JJ` ; respecter les bornes affichées. Le 5 novembre 2026, la date testée était `2026-11-12`.
7. Créer la clé, puis copier sa valeur une seule fois dans le terminal ou un fichier privé. Masquer cette valeur lors d'un enregistrement public.

Attention au raccourci d'accueil : Manage API keys peut ouvrir l'ancienne page sans onglet Subscriptions. Passer par le menu latéral **Gen AI studio → API keys**.

À dire : « Je n'ai ni le GPU ni le serveur à administrer. Ma clé personnelle hérite de l'accès et de la limite de cette souscription. »

Pour stocker la clé sans l'afficher :

```
mkdir -p secrets
chmod 700 secrets
read -rsp 'Clé MaaS : ' DEMO_KEY; echo
(umask 077; printf '%s' "$DEMO_KEY" > secrets/demo-user.apikey)
unset DEMO_KEY
```

6 — DATA SCIENTIST : faire l'appel réel

Dans un terminal utilisateur, avec Python, curl et `scripts/client.py` :

```
export MAAS_URL=https://maas.apps.<domaine-de-la-sandbox>
python3 scripts/client.py --key-file secrets/demo-user.apikey
```

À montrer : HTTP `200`, la réponse Qwen et le bloc `usage`. Identifier `prompt_tokens` (entrée), `completion_tokens` (sortie) et `total_tokens` (somme).

À dire : « Cet appel utilise ma clé, pas le token de l'administrateur. Le modèle découvert appartient à la souscription qui m'a été accordée. »

Le client effectue ces deux requêtes : `GET /v1/models`, puis `POST /v1/chat/completions`. Pour montrer le contenu pédagogique du POST :

```
{
  "model": "<identifiant exact retourné par /v1/models>",
  "messages": [{"role": "user", "content": "Explique en une phrase à quoi sert une API."}],
  "max_tokens": 64
}
```

L'identifiant servi par l'assistant UI peut différer de celui du manifeste GitOps. Toujours reprendre l'ID retourné par `/v1/models` ; ne pas choisir arbitrairement le premier modèle d'un catalogue contenant plusieurs projets.

7 — ADMIN : montrer les tokens réellement consommés

Avant une série d'appels, ouvrir **Observe & monitor → Dashboard → MaaS démo** et noter le compteur de tokens. Cet onglet est ajouté par notre kit et interroge de vraies métriques.

Côté data scientist :

```
python3 scripts/client.py --key-file secrets/demo-user.apikey --count 5
```

Revenir côté admin, attendre le cycle de collecte (30 à 60 secondes), puis **Refresh**.

À montrer : le compteur **Tokens consommés** augmente, les appels autorisés augmentent et la courbe porte le nom de la souscription/modèle. Comparer l'écart du compteur à la somme des `total_tokens` des réponses.

À dire : « Je peux suivre l'usage de ce service par souscription. Ces nombres proviennent du contrôle MaaS, pas de données dessinées pour la démonstration. Les compteurs repartent de zéro au redémarrage du composant. »

Le dashboard natif **Usage** présente une incompatibilité de labels sur cette build. Utiliser l'onglet **MaaS démo** pour cette démonstration et annoncer qu'il s'agit d'une vue Perses ajoutée par le dépôt. Aucun détail par utilisateur ou par clé n'est inventé.

Contrôle de secours admin : `make usage` lit les mêmes compteurs à la source.

8 — BONUS : prouver que la limite est appliquée

Préparer `demo-low-quota` (50 tokens/minute) avec le même groupe et le même modèle, puis créer en tant que `demo-user` une seconde clé liée uniquement à cette souscription. Le manifeste est `manifests/govern/low-quota.yaml`.

```
python3 scripts/client.py --url "$MAAS_URL" \
  --key-file secrets/demo-user.ratelimit.apikey --count 5 --expect-429
```

Preuve : appel accepté, puis **429**. Le premier appel peut dépasser le petit budget avant que la réponse soit comptabilisée. Revenir sur **MaaS démo** → **Appels limités** après collecte.

À dire : « La limite est appliquée à la consommation. Ma clé principale reste dans sa souscription de démonstration. »

Option révocation : révoquer une clé de test dans API keys, puis refaire un appel ; un refus est attendu. Ne pas annoncer un code unique 401/403 sans le constater sur la version utilisée. Sur cette sandbox, le cache d'autorisation dure 60 secondes : attendre sa propagation avant de conclure.

9 — Phrase de fin

« Nous avons choisi un modèle, exécuté ses poids dans OpenShift, publié une API gouvernée, donné une souscription à une équipe et laissé le data scientist créer sa clé. L'administrateur peut ensuite mesurer les tokens consommés. Le dépôt reconstruit cette démonstration sur une sandbox compatible. »

Aide-mémoire du présentateur

ADMIN : OpenShift → OpenShift AI → Projects → Create project → Deployments → Deploy model → URI → Generative → llm-d → 1 GPU → Publish as MaaS → Deploy.

ADMIN : Settings → MaaS governance → Create subscription → groupe + modèle + 2 000/min → Authorization policies → demo-access.

DATA SCIENTIST : fenêtre privée → connexion demo-user → API keys → Subscriptions → Create API key → appel → réponse et tokens.

ADMIN : retour à la fenêtre normale → Observe & monitor → Dashboard → MaaS démo → Refresh → augmentation des compteurs.